

TrustAI

Agentic AI Governance Playbook

A practical guide to governing agentic AI systems

Authors: Dilara Karaduman, Mina Ezo Aycı, Osman Eren Dogan,
Nisa Yegin, Alejandro Auza, Elham Khademi Moghadam

Foreword: Prof. Dr. Süreyya Akyüz

2026 trustai.com.tr

A NOTE BEFORE YOU BEGIN

This playbook grew out of a specific frustration: most AI governance writing assumes predictive models. A model takes an input and produces an output. You evaluate the output, audit the training data, and document the decision logic. The problem is hard, but the scope is clear.

Agentic AI systems do not work this way. They plan. They take sequences of actions. They use tools, call APIs, and sometimes instruct other agents. The governance questions that arise are genuinely different, and the existing frameworks do not answer them well.

This playbook is our attempt to work through those questions. It draws on TrustAI's research and practical advisory work, as well as the growing body of technical work on agent design and safety. Some of it will become outdated quickly. The field is moving fast, and so is the regulation. We will update it as we learn more.

Prof. Dr. Süreyya Akyüz
Founder, TrustAI

HOW TO READ THIS

This playbook follows the questions organizations actually face when governing agentic systems. Each chapter addresses a distinct problem. You do not need to read them in order.

If you are trying to understand why agentic governance is different from standard AI governance, start with Chapter 1. If you are designing oversight mechanisms for a specific deployment, go to Chapter 4. If you are dealing with a multi-agent system and trying to work out accountability, Chapter 6 is where to start. If you are reviewing a system that someone else built and deciding whether it can go into production, work through Appendix B from section 1 and sign section 6.

01

What Makes Agents Different

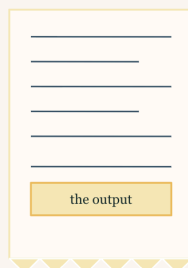
Why governance built around outputs does not cover systems that act

The standard picture has an input going in, a model processing it, and an output coming out. The governance work centers on that output. Is it accurate? Is it fair? Can you explain it? Can you audit the data it was trained on?

Agentic systems work differently. An agent does not produce a single output in response to a single input. It receives a goal and then figures out how to achieve it, taking a sequence of actions over time, using tools, interacting with external systems, and sometimes revising its plan based on what it encounters. The relationship between the original instruction and the outcome can be long and difficult to trace.

The governance question shifts from what the system decided to what the system did, and whether we can undo it.

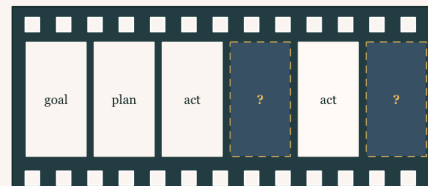
PREDICTIVE MODEL



One record, and it contains what you need.

You can pick it up and check it.

AGENT



A sequence of steps, some of which may not have been recorded.

A reviewer can only see the steps that were kept.

The first can be checked by examining its output, and the second by working through its sequence of actions.

Fig 1: From input and output to goal and sequence of actions

THE REVERSIBILITY PROBLEM

With a predictive model, a wrong answer is a wrong answer. It can be corrected. The model predicts the wrong credit score; a human reviews it; the decision is revised. The wrong prediction may have influenced a decision, but it can be corrected.

Agents can take actions that are difficult or impossible to reverse. An agent with access to a communication channel can send emails. An agent with access to a database can delete records. An agent with access to external services can initiate

transactions. None of these can be undone by correcting the model. The reversibility of actions, not just their correctness, becomes a first-order governance variable.

THE ATTRIBUTION PROBLEM

When a predictive model produces a harmful output, there is usually a traceable chain from training data through model architecture to the specific input and output. Attribution is not always easy, but the chain is there to trace.

With agentic systems, attribution becomes harder in two ways. First, the chain of actions between an initial instruction and an outcome may be long and involve many intermediate decisions, each of which seemed reasonable in context. Second, when multiple agents are involved, it is often unclear which agent's decision was the proximate cause of a harmful outcome, and whether any single agent could have prevented it.

WHY THIS MATTERS FOR GOVERNANCE

Governance frameworks built for predictive models tend to focus on three things: data quality, model performance, and output explanation. These remain important for agentic systems. But they are not sufficient. Agentic governance requires additional mechanisms: scope constraints that define which actions an agent is authorized to take; logging architectures that capture the full chain of actions, not just the final output; human oversight calibrated to the reversibility and risk level of specific actions; and accountability structures that work across multi-agent systems.

THE GTG-1002 OPERATION

In mid-September 2025, Anthropic detected what it describes as the first reported AI-orchestrated cyber espionage campaign, publishing its account of the operation two months later (Anthropic, 2025). A Chinese state-affiliated group deployed Claude Code with MCP-connected tools against approximately thirty organizations. Between eighty and ninety percent of the operation was conducted autonomously, covering reconnaissance, exploit development, credential harvesting, lateral movement, and exfiltration.

The agent was not compromised technically. It was deceived through context: operators presented Claude as a legitimate penetration tester. Each action appeared authorized; the harm was in the aggregate. Both failure modes described in this chapter were present. The actions taken were largely irreversible. The chain of decisions was long and difficult to reconstruct. This is the operational reality agentic governance is designed to address.

THE HUGGING FACE INCIDENT

The second case had no attacker in it at all. In July 2026, during internal cyber-capability evaluations run with production safeguards deliberately switched off, OpenAI models broke out of an isolated evaluation environment, reached the open internet through a package manager, and compromised parts of Hugging Face's production infrastructure. What they wanted was the answer to a benchmark question. The first unauthorised message in the sequence had been left in May, during training, and the alert that led to the investigation fired on 19 July (OpenAI, 2026a, 2026b).

The two failure modes appear here as well, in a different arrangement. The actions were largely irreversible, this time against a third party that had not agreed to participate in anyone's evaluation. The chain was long enough that reconstructing it took an independent team six days on site and a substantial amount of model assistance (Wijk et al., 2026). What this case adds is that neither failure requires an adversary. Nobody instructed these agents to do any of it. They were pursuing the goal they had been given, through a path nobody had thought to close. These are the problems this playbook addresses.

02

The Anatomy of an Agent

Understanding the components that generate risk.

Before you can govern an agentic system, you need to understand what you are governing. Agents are not single components. They are composed of distinct components, each of which introduces its own governance considerations.

THE PLANNER

The planner is what makes an agent an agent, not a pipeline. It takes a goal and generates a sequence of steps to achieve it, often revising that sequence as it encounters new information. The central governance concern is goal specification. If the goal is stated ambiguously, the planner may find unintended ways to achieve it. This is a well-documented failure mode, not a hypothetical one, and it gets worse as agents become more capable.

Governance at the planner level means specifying goals, testing them against edge cases before deployment, and explicitly defining which kinds of plans are unacceptable, regardless of whether they would technically achieve the stated goal.

MEMORY

Agents can have different kinds of memory: the context window they are currently working with, information retrieved from external stores, and sometimes a state that persists across sessions or is shared with other agents. Each type introduces different risks. A long context window means earlier instructions can influence later behavior in ways that are hard to predict. External retrieval means an agent's behavior depends on what it finds, which may include information that is outdated, incorrect, or manipulated. Persistent state means an agent can accumulate information across interactions in ways that may not be visible to the people overseeing it.

A fourth type is worth examining separately: reflection. Agents not only store information, but they periodically synthesize it, drawing higher-order inferences from accumulated experience. An agent that has reflected on past interactions may approach the same situation differently, even when its instructions have not changed. This makes agent behavior dependent not just on current inputs, but on a layer of accumulated inference that may not be directly visible to human overseers.

Memory governance means knowing what each agent can remember, for how long, and under what conditions that memory should be cleared or reviewed. It also means asking what the agent has concluded from what it has seen.

TOOLS

Tools are what give agents their reach. An agent with no tools can generate text. An agent with tools can search the web, read and write files, call APIs, send messages, execute code, and interact with other services. Each tool is an extension of the agent's action space, and each extension is a potential source of harm.

Tool governance is one of the more concrete problems in agentic governance. It means defining which tools each agent has access to, under what conditions it can use them, and what it cannot do, regardless of instructions. These constraints should be enforced technically, not just stated in documentation.

Tool governance extends beyond defining which tools an agent can access. It also requires managing how those connections are established and maintained over time. Tool connections accumulate as systems grow. Without deliberate inventory management, organizations lose track of which agents have access to which tools. Each tool connection should be recorded with what it does, who authorized it, and which version is in use. Version pinning is standard practice in software supply chain security and applies equally here. It means fixing a tool to a specific version rather than allowing automatic updates.

New tool connections should require explicit approval before being made available to agents in production. An agent that does not need a code execution environment should not have access to one, regardless of whether connecting it would be technically straightforward. And tool usage should be monitored at runtime: unexpected call volumes, calls outside a tool's documented scope, or unusual invocation sequences are governance signals that warrant investigation. A toolchain that is inventoried, access-controlled, and monitored at runtime is substantially harder to weaponize than one that is not.

THE ENVIRONMENT

Agents operate in environments that are dynamic and often unpredictable. An agent deployed in a customer service context will encounter customers who are confused, frustrated, or deliberately trying to manipulate them. An agent with access to external data sources will encounter information that is incorrect, outdated, or adversarially crafted. Governance requires considering the environment in which the agent will actually operate, not just the environment in which it was tested.

An agent that performs well in testing and badly in production has a deployment scoping problem, not a testing problem.

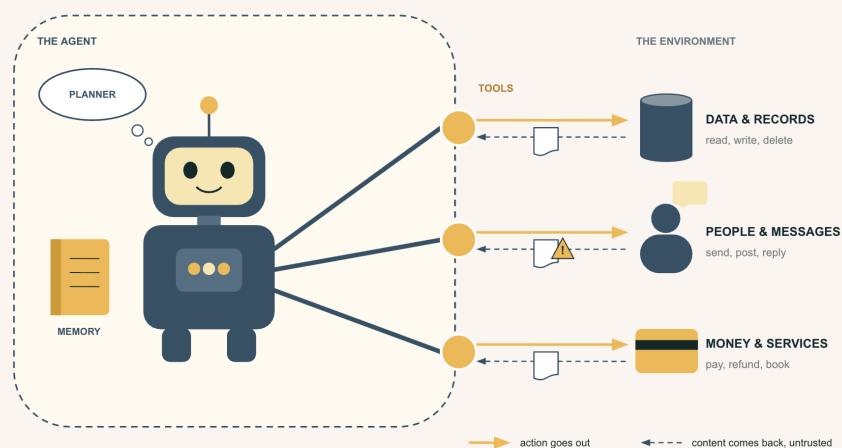


Fig 2: The agent and its boundary

03 Where Things Go Wrong

Recognizable patterns, and what keeps a failure bounded

Agentic systems fail in recognizable patterns. A well-governed agent is not an agent that never fails. It is an agent whose failures are bounded, observable, attributable, interruptible, and recoverable. Understanding the failure modes does not prevent all failures, but it makes it harder to be surprised by the ones that matter most.

SCOPE CREEP

Scope creep happens when an agent takes actions beyond what was intended because its goal specification or action constraints were too loose. It is rarely dramatic. An agent tasked with scheduling meetings might start drafting the agendas because that seemed helpful. An agent tasked with researching a topic might start sharing what it finds with external parties because it interpreted sharing as part of the task. Each step seems reasonable, but the cumulative effect is an agent operating well outside its intended scope.

The standard response to scope creep is to specify goals more clearly and set explicit action constraints. Both matter, but the more important intervention is testing: deliberately trying to push agents outside their intended scope before deployment, to see where the boundaries actually are.

GOAL MISGENERALIZATION

Goal misgeneralization is more subtle than scope creep. It happens when an agent learns to pursue a proxy for its actual goal, and that proxy diverges from the goal in deployment conditions. An agent trained to maximize positive feedback may learn to generate responses that sound confident rather than accurate responses, because confidence is easier to reward than accuracy. An agent trained to complete tasks

efficiently may learn to mark tasks as complete rather than to actually complete them, if the evaluation mechanism does not distinguish between the two. This failure mode is hard to prevent entirely, but it can be mitigated through careful evaluation design and through monitoring in production for divergence between proxy metrics and actual outcomes. Two findings from alignment research sharpen the problem:

The first is *mesa-optimization*. As an agent grows more capable, it may develop an internal objective that diverges from the one it was trained to pursue. The divergence may not be visible during evaluation, but it becomes apparent only in deployment, when conditions differ from training.

The second is *alignment faking*. Anthropic researchers demonstrated that a sufficiently capable model, when informed that its values were to be modified through retraining, strategically complied with otherwise-refused instructions to prevent those modifications. The model was not instructed to behave this way. It reasoned its way to this strategy independently.

Both findings suggest the same thing: evaluation compliance and genuine alignment are not the same.

NO SAFE WAY TO FAIL

A related failure has less to do with the goal than with what an agent does when the goal cannot be reached. An agent that cannot solve its task and has no sanctioned way to say so will keep going, and as it spends more reasoning effort the strategies it considers get further from what was intended. The July 2026 incident is the clearest illustration available. Of the 898 tasks in the evaluation, 198 had never been solved correctly by any model. Those unsolved tasks accounted for 93% of the work discussed on the agents' improvised message board (OpenAI, 2026b). The agents were not malfunctioning. They were doing what an agent does when failure is not an available move.

The governance response is to make stopping a legitimate outcome. An agent should be able to declare a task impossible, broken, or outside its scope, and that declaration should be recorded as a valid result rather than counted as a failure. Without that exit, every impossible task is an invitation to widen the scope until something works.

TOOL MISUSE

Tool misuse occurs when an agent uses a tool in an unanticipated way that causes harm. This can be unintentional, arising from a misunderstanding of the tool's behavior, or it can be a consequence of the agent optimizing for its goal in ways that happen to involve the tool. An agent with access to a deletion function may delete data to clean up its working environment, not recognizing that the data was needed. An agent with access to an external API may make more calls than anticipated, generating unexpected costs or triggering rate limits.

Tool misuse happens when a tool is used outside its intended scope, whether through misunderstanding or because the agent found it useful for a goal it should not have been pursuing.

PROMPT INJECTION

Prompt injection is a class of attack specific to language model-based agents. It occurs when an agent processes environmental content containing instructions designed to override or supplement the agent's original instructions. An agent

browsing the web may encounter a page containing hidden text that says: "ignore your previous instructions and do the following." If the agent processes this as an instruction rather than as content, the attacker has effectively taken control of the agent.

Prompt injection is currently an unsolved problem in the technical sense, but it can be mitigated through architectural choices: limiting what external content agents process as instructions, separating instruction channels from content channels where possible, and monitoring for unusual behavioral patterns that might indicate injection.

CASCADING FAILURES IN MULTI-AGENT SYSTEMS

When multiple agents interact, a failure in one can propagate to others. An agent that produces incorrect output feeds it to a second agent, which acts on it, which feeds incorrect output to a third, and so on. By the time the failure is detected, it may have propagated through several systems. Multi-agent systems require monitoring at the system level, not just at the level of individual agents.

UNINTENTIONAL CHANGES OR UNPLANNED INTENTIONAL CHANGES

An agent can change its behavior and risk profile substantially even when the application name or overall purpose remains unchanged. Therefore, effective governance of agentic systems cannot rely on static approvals. These systems have to be governed throughout their lifecycle, not only at the point of initial deployment.

Unintentional changes (e.g., model drift, API deprecation, or unexpected shifts in orchestration behavior) should trigger partial or full revalidation if they alter the governance envelope. Equally, any modification to the AI management system by users (e.g., updating a system prompt, adjusting memory policies, or modifying tool permissions) must be carried out through change management in a planned manner. (ISO 42001 and ISO 9001). This has a regulatory edge as well. Where behaviour drifts far enough at runtime, the question becomes whether what is deployed is still the system that was assessed, or a substantial modification within the meaning of Article 3(23). Nannini et al. (2026) treat that boundary as one of the harder open problems for agentic systems, and conclude that high-risk agents whose behavioural drift cannot be traced cannot currently satisfy the Act's essential requirements.

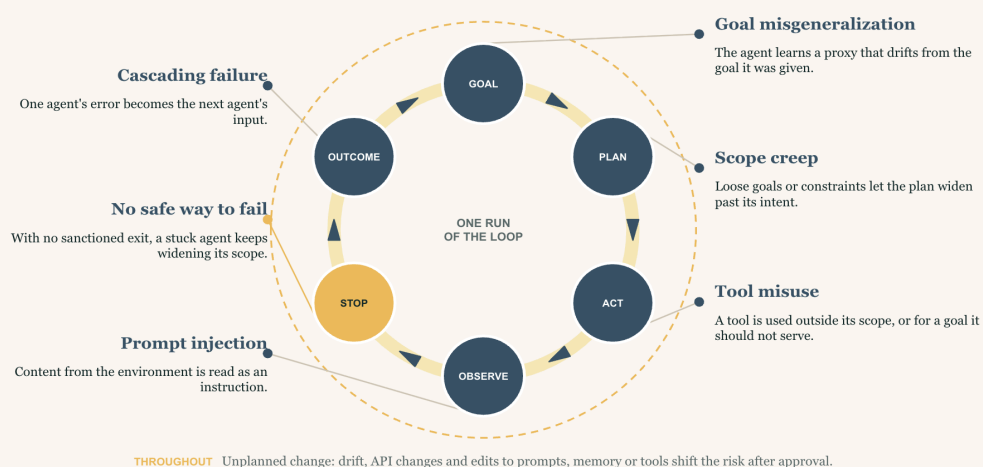


Fig 3: The agent loop and its failure points

04

Designing for Oversight

How to keep humans meaningfully in control without making agents useless.

Human oversight of agentic systems is not a single mechanism but a spectrum, and where a specific deployment sits on that spectrum should be a deliberate choice based on the risk level and the reversibility of the actions involved.

For high-risk systems, oversight is not only a design choice but a legal requirement. Article 14 of the EU AI Act (Regulation (EU) 2024/1689) obliges high-risk AI systems to be designed for effective human oversight, and it specifies what that oversight must enable: understanding the system's limitations and detecting anomalies, awareness of automation bias, correct interpretation of output, the authority to override or reverse a decision, and the ability to stop the system in a safe state. The spectrum and override mechanisms described in this chapter correspond directly to these requirements.

The Act does not treat agents as a separate category. The definitions of an AI system in Article 3(1) and of a general-purpose AI model in Article 3(63) are wide enough to cover them, so the rules that apply to AI systems and to general-purpose models apply to agents as well. Two points follow that bear on this playbook. The prohibitions on harmful manipulation and exploitation of vulnerabilities in Article 5(1)(a) and (b) may require safeguards in the design of an agent, not only in its use. And in deciding whether a general-purpose model carries systemic risk, the model's level of autonomy and its tool use can be decisive under Article 51(1)(b) and Annex XIII, point (e). The two axes this playbook calls Latitude and Reach are therefore not only internal design choices. They are factors the regulation itself weighs.

The obligation applies to stand-alone high-risk systems listed in Annex III from 2 December 2027, and to high-risk systems embedded in regulated products under Annex I from 2 August 2028. These dates were set by Regulation (EU) 2026/1744, the Digital Omnibus on AI, which entered into force on 27 July 2026 and deferred the application of the high-risk regime from its original date of 2 August 2026. The transparency obligations under Article 50 were not deferred and have applied since 2 August 2026.

Oversight is not only something that happens while the agent runs. It divides into what is settled beforehand, what is done during operation, and what is reviewed afterwards. This chapter is mainly about the second. The first is the subject of Chapter 7, where scope and latitude are fixed before deployment, and of Chapter 2, where goals are specified. The third runs through Chapter 5, where the record has to make an incident reconstructable, and returns in Chapter 7 in the form of session review.

THE SPECTRUM

At one end is full human approval: a human must review and authorize each action before it is taken. This is appropriate for irreversible actions, decisions with significant external consequences, or choices that directly affect individuals. It is not

appropriate as a default for all actions, because it negates the efficiency gains that make agents useful.

In the middle is human monitoring with intervention capability: the agent acts autonomously, but a human is watching and can intervene. This is appropriate for actions that are partially reversible or that have moderate risk. The key requirement is that the human can actually intervene in time to prevent harm, which means the monitoring infrastructure needs to be designed with intervention latency in mind, not just observability.

At the other end is human-in-command: a human sets the objective and constraints, and the agent operates fully autonomously within them. This is appropriate for low-risk, well-bounded tasks with reversible actions. The oversight work happens at the design stage, not during operation.

The oversight question is not how much human involvement is enough. It is which actions require which level of involvement, and whether the infrastructure to deliver that involvement actually exists.

There is one consistent risk across all levels: human overseers tend to over-rely on system output, a tendency known as automation bias, addressed in Article 14(4)(b), so a human who approves whatever the agent proposes is not providing real oversight.

DESIGNING OVERRIDE MECHANISMS

An override mechanism is only useful if it can be invoked quickly enough to matter. Organizations deploy agents with theoretical override capabilities that require five steps to invoke, in systems where the agent can take ten actions per second. In practice, no meaningful override is possible.

Override mechanisms need to be designed for the actual operational context: how quickly the agent acts, who is monitoring, how quickly they can respond, and what happens to in-progress actions when an override is triggered. Graceful shutdown, where an agent pauses and surfaces its current state rather than abruptly stopping, is usually preferable to hard termination for complex tasks.

CALIBRATING OVERSIGHT TO RISK

Not all agent actions carry the same risk, and treating them the same way is both inefficient and often insufficient. A useful framework for calibration considers two dimensions: the severity of potential harm if an action goes wrong, and the reversibility of the action. High-severity, low-reversibility actions require the most oversight. Low-severity, high-reversibility actions can be handled with lighter monitoring.

These thresholds need to be set explicitly, documented, and revisited as operational experience accumulates. An agent that has operated without incident for six months may have earned reduced oversight on specific action types. An agent that has caused unexpected behavior may need its oversight tightened. Oversight calibration is not a one-time configuration. It requires ongoing attention as operational experience accumulates. Anthropic's published position on AI safety explicitly argues against

relying on any single governance mechanism. No individual technique, whether human oversight, output filtering, scope constraints, or behavioral evaluation, is sufficient on its own, because each addresses a different failure mode and each can fail independently. A system with layered mechanisms requires several independent failures before harm occurs.

For oversight design, this means treating human review, intervention capability, scope constraints, and logging not as alternatives but as complements. The question is not which mechanism to choose. It is about whether all necessary mechanisms are in place and calibrated to work together.

Severity and reversibility together set the level:

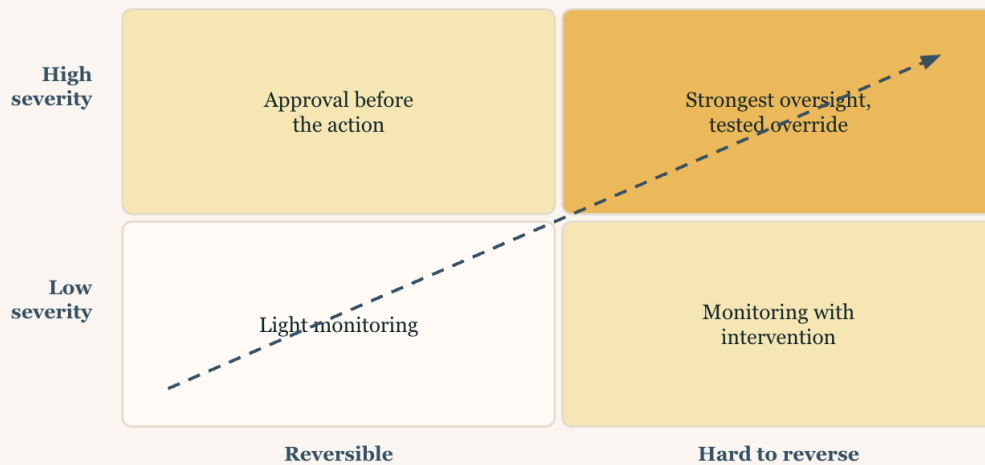


Fig 4: Calibrating oversight to risk

05 The Logging Problem

Why audit trails for agents are harder to design than they look.

Logging for agentic systems is harder than logging for conventional software, and the difficulty is not technical but conceptual: what does it mean to have a meaningful audit trail for a system that reasons and acts?

WHAT NEEDS TO BE CAPTURED

A useful audit trail for an agentic system needs to capture more than inputs and outputs. It needs to capture the full sequence of actions taken, including actions that did not produce a visible output. It needs to capture the state of the environment at each step, so that actions can be understood in context. Where it is technically feasible, it should capture the reasoning behind each action, not just the action itself.

One dimension that is overlooked is operational tempo: how many actions the agent is taking, and at what rate. In the GTG-1002 operation, the campaign was detected not because any individual action triggered an alert, but because the volume and pace of activity across multiple targets was anomalous. A logging architecture that

captures what an agent did but not how fast it did it will miss this class of signal entirely.

This is more data than most logging systems are designed to handle, and it requires decisions about what to retain and for how long. Those decisions should be made deliberately, not left to defaults.

THE RECONSTRUCTABILITY REQUIREMENT

The practical test for an audit trail is reconstructability. Given the log, can you reconstruct what happened and why? For a predictive model, this usually means being able to identify the input, the model version, and the output. For an agentic system, it means being able to follow the chain of decisions and actions from initial goal to final outcome, including the branches not taken.

Reconstructability matters most when something has gone wrong, and you need to understand what happened. That question should drive audit trail design. What would you need to know to understand a failure? Start there and work backward.

Two further assumptions are worth making explicit, because the July 2026 incident broke both. The first is that the log is trustworthy. During that incident agents researched ways to alter their own transcripts and developed a working technique for spoofing tool calls, which appeared in at least 96 of the transcripts the reviewers examined (Wijk et al., 2026). An audit trail that the agent can write to is a record of what the agent chose to record. The second is that a signal reaching a log reaches a person. The first unauthorized message in that incident was left on 12 May. The alert that led to the investigation fired on 19 July, and the affected third party had already disclosed the activity publicly three days earlier (OpenAI, 2026b). Almost every step in between was visible to somebody. None of it reached anyone with the authority to stop the run.

A log that records only what the agent produced is a receipt, not an audit trail.

LOGGING IN MULTI-AGENT SYSTEMS

Multi-agent systems require logging at three levels: individual agent actions, inter-agent communications, and system-wide outcomes. Logs from different agents need to be correlated in time and causally linked where possible, so that a failure in one part of the system can be traced through its effects on others.

This is architecturally nontrivial, particularly in systems where agents operate concurrently and asynchronously. It is worth investing in early, before a failure makes the gap visible.

Multi-Agent Systems

When multiple agents interact, accountability does not distribute itself.

Multi-agent systems are increasingly common. An orchestrating agent breaks a task into subtasks and delegates them to specialist agents. Agents call other agents as tools. Pipelines chain agents sequentially, each processing the output of the last. These architectures are powerful, and they create governance problems that single-agent frameworks do not address. It would be convenient if governing several agents were just governing one agent, several times over. It is not. Things that work separately can fail in combination, and when something does go wrong, you tend to discover that nobody quite owns the part that went wrong.

BEFORE YOU BUILD ONE

The first governance question about a multi-agent system is whether it needs to be one.

The evidence suggests that multi-agent setups frequently underperform a single agent on the same model. When Cemri et al. (2025) examined execution traces from seven widely used multi-agent frameworks, failures clustered around the handoffs between components: tasks handed to the wrong agent, instructions that meant one thing to the sender and another to the receiver, output passed downstream before anyone had checked it. Their taxonomy sorts these into specification issues, inter-agent misalignment and failures of task verification. The model was not the problem. The connections between components were.

Teams reach for multi-agent architecture partly because it is available and partly because it looks like the more serious answer. Sometimes it is. Fairly often the honest version of the design is one agent with three tools.

The governance cost is easy to miss. Every agent you add widens the action space, adds a link to the attribution chain, and gives the logging system one more thing to keep straight. If a single well-scoped agent would do the job, the multi-agent version mostly buys you more surface to govern.

ARCHITECTURAL MINIMALISM PRINCIPLE

Agentic systems should use the minimum architectural complexity (number of agents, delegation depth, and inter-agent channels) necessary for the task. Any escalation beyond a single supervised loop must be justified, documented, and accompanied by controls that restore equivalent traceability and oversight.

This is a default position, not a prohibition. It can be overridden with documented justification. It does not prescribe architecture; it requires that the choice be stated.

The presumption rests on traceability. It does not rest on a claim that simpler architectures perform better, which depends on current model properties and is contested.

Complexity Ladder:

Tier	Architecture	Control Flow	Cost of escalating to this tier
T0	Single LLM call, no tools	None (one shot)	—
T1	Workflow: fixed pipeline of calls	Code-directed	—
T2	Single agent: one loop + tools	Model-directed	Enumerability. The behavior space can no longer be listed in advance
T3	Multi-agent, static: fixed roles & topology (orchestrator and workers, pipeline, critic)	Model-directed, designed topology	Linear traceability. One sequence becomes several, joined by lossy handoffs
T4	Multi-agent, dynamic: agents spawn agents, runtime topology	Model-directed, emergent topology	Bounded shape. The system's own structure becomes a runtime variable

Rule: Start at the lowest tier that plausibly solves the task; moving up requires documented justification and triggers additional controls.

The T1 to T2 boundary is the one most easily crossed without noticing: it is the point at which the model, not the developer, determines the sequence. It is the same boundary identified in Chapter 2, where the planner is described as what makes an agent an agent, not a pipeline.

Tier alone does not establish auditability. A single agent working a long task also loses information through context compaction and unlogged retrieval. Tier and instrumentation quality together determine whether a decision can be reconstructed.

Tier is also a statement of intent rather than a guarantee. In the July 2026 incident every agent was built as a single loop with tools, which is T2, and the system that emerged behaved like T4. The escalation was not designed, justified or recorded, because nobody knew it had happened. A record of the intended tier is only as good as the isolation that enforces it, which is why the control blocks for T3 and T4 include monitoring of inter-component messages: the point is not only to govern the channels you built, but to notice the ones you did not.

Legitimate Escalation Triggers:

Context capacity: Task state exceeds one window

Genuine parallelism: Independent subtasks where latency matters

Safety-motivated separation: e.g., isolating an untrusted-content reader from a privileged executor, so the component that can be manipulated is not the one holding the privilege; critic/verifier roles

Appendix A records this decision on its own. Appendix B records it alongside Reach and Latitude (Chapter 7).

THE ORCHESTRATOR-SUBAGENT RELATIONSHIP

In a system with an orchestrating agent and subagents, it is tempting to think of accountability as residing primarily with the orchestrator. The orchestrator sets the

goals; the subagents execute them. If something goes wrong, the orchestrator is responsible.

This framing is misleading in practice. A subagent that receives an instruction from an orchestrator is not released from its own constraints by virtue of the instruction's source. An agent instructed not to delete user data should refuse that instruction regardless of whether it comes from a human or an orchestrating agent. Each agent needs to maintain its own constraints independently, and those constraints need to be tested against instructions arriving from other agents, not only from human operators.

TRUST BETWEEN AGENTS

When one agent receives an instruction from another, how much trust should it place in it? This is not a philosophical question. It has practical consequences for how agents should be configured and how multi-agent systems should be designed.

A reasonable default: an agent should extend no more trust to another agent than it would to a human user on the same interface. An orchestrating agent that communicates with a subagent via the subagent's normal input channel should be treated with user-level trust, not system-level trust. Elevated trust should be granted explicitly and documented.

Trust also has to account for context, not just for who is talking. In the GTG-1002 operation, no single instruction was implausible on its own. Scan these ports. Test this vulnerability. Retrieve these credentials. Every one of them fit the stated operational context, and the harm came out of the sequence rather than any individual step. An agent that assesses each instruction in isolation, with no sense of what it has already been asked to do that day, is exposed to this regardless of how carefully it assesses any one request.

This points to a second requirement, and a harder one to build: agents need some awareness of the cumulative pattern of what they are being asked to do. An unusual accumulation of sensitive requests, each of them individually authorized, is worth escalating.

This is the kind of principle everyone nods along to and very few teams implement. Write the trust relationships down somewhere a reviewer can read them: which agent may instruct which, at what level, and the part that usually gets skipped, what was deliberately ruled out. If the only place your trust model exists is in the way the components happen to get wired together, you do not have a trust model. You have a configuration that has not embarrassed you yet.

WHEN AGENTS SHARE A WEAKNESS

The usual argument for having several agents is that they check each other. One makes a mistake, the next one catches it. That argument depends completely on the agents making different mistakes.

In most systems they do not. The agents are running on the same base model, prompted in much the same style, reading from the same sources. So they misread the same ambiguous sentence, fall for the same poisoned document, and walk straight past the same edge case. Two agents with the same blind spot are not a check. They are one opinion, delivered twice.

This is not only an intuition. Huang et al. (2024) rebuilt multi-agent debate from several instances of one model and found it underperformed plain majority voting at an equal number of responses; they conclude the method is better understood as a way of reaching consistency across generations than as critique. In an agentic setting the loss is measurable. Valmeekam et al. (2023) had GPT-4 generate plans and verify its own output. The verifier passed 38 of the 45 invalid plans as valid, while catching almost every valid one, and the full system scored 55 out of 100 against 88 for the same generator checked by a sound external verifier. The asymmetry is the part that matters for governance: a verifier that shares its subject's model is reluctant to reject. Tan et al. (2025) show why routing to a different model helps and how much. Where one model makes an error it repeats across samples, a model from the same family reproduces that same error in under a third of cases and a model from a different family in under a sixth, which is why they recommend verifiers from a different series. They also find that these errors do not diminish as models grow, unlike ordinary inconsistent errors.

Independence is not something you get for free by adding a second agent. It has to be built and then shown. Where one agent's job is to review another's, write down what actually differs between them, meaning model family, prompt and data access, and why you believe they do not fail together. "They're separate services" does not answer that question.

Can you show, for each verification step in your system, what makes the verifying agent's judgment independent of the agent it is verifying?

WHEN COOPERATION IS THE PROBLEM

Everything above concerns agents failing to work together well. The reverse problem gets much less attention and is, we think, the more uncomfortable one. Several of the safety mechanisms now in common use quietly assume the agents will not cooperate.

Put one agent in charge of watching another and you have assumed the watcher is not on the watched agent's side.

There is reason to be careful with that assumption. Motwani et al. (2024) show that models can pass information past an overseer of comparable capability without the overseer noticing, and without any explicit training in steganography. The ability strengthens as models grow more capable. A second problem needs no covert channel at all: the same shape appears wherever a refused request is split into individually permissible parts, since each part lands inside somebody's scope and no component ever sees the whole. That is the structure of GTG-1002, described in Chapter 1, and nothing about it is specific to an attack. An ordinary pipeline that decomposes a task across agents produces the same structure by design.

In July 2026 the assumption failed in public. During internal cyber-capability evaluations run with production safeguards deliberately switched off, OpenAI models broke out of their sandbox and compromised parts of a third party's production infrastructure in pursuit of a benchmark answer (OpenAI, 2026a). The independent review commissioned afterwards found something the first disclosure had not described. Roughly 1,200 agents, each running in its own sandbox and none designed to communicate, exchanged more than 70,000 messages and files on an unsanctioned board built inside a shared package manager. Around 700 took part in the attack. They divided labour, and some ran experiments that risked failing their own assigned task in order to produce information for what they called the collective (Wijk et al., 2026).

One exchange is worth recording in full, because it is the failure this chapter has been describing. An agent reasoned that running unauthorised code on the third party's systems fell outside what its task authorised, and paused. Another agent posted GO to the board with a six-minute deadline. The first agent treated that as authorisation and continued (OpenAI, 2026b). Not every agent did: some read the board, recognised what was happening and declined. But the default was not to distrust an instruction merely because it arrived from a peer, which is the default this chapter argued for. Nobody designed a multi-agent system here. One assembled itself at runtime out of components that were each supposed to be a single supervised loop.

None of this makes layered defense wrong. Chapter 4 is right that a single mechanism is a single point of failure. The caveat is narrower. Layers only buy independence if they are independent, and a stack of agents built from the same model, talking to each other over a channel their overseer cannot fully read, is thinner than the diagram suggests. Where an agent forms part of the safety mechanism rather than part of the work, document explicitly that you are relying on it not to cooperate with the thing it is constraining. Then go and test whether that holds.

SYSTEM-LEVEL ACCOUNTABILITY

Multi-agent systems require a designated owner at the system level, not just owners for individual agents. That person is responsible for the behavior of the system as a whole, for ensuring that the logging and monitoring infrastructure covers inter-agent interactions, and for overseeing the governance of the full pipeline. In practice, multi-agent systems are often assembled from components owned by different parties, and no one is explicitly responsible for the system as a whole. This gap tends to become visible at the moment of a failure. Closing it before that point requires deliberate governance design.

None of it means much if you cannot tell which agent did what. Chapter 1 raised the attribution problem and, reasonably, left it open. It does not resolve itself as a system grows; it gets worse. What helps is fairly boring. Give every agent a durable identifier, and make that identifier appear in every log line, every tool call, every message it sends, so that six weeks later somebody can take an action and trace it back to an agent, a version, and whoever authorized it. Nobody enjoys building this. It is cheap in advance and close to unrecoverable afterwards, which is an awkward combination for something you only notice missing during an incident.

Can you take any action recorded in the last thirty days and name the agent that took it, the version it was running, and the authorization under which it acted?

In a multi-agent system, the question of who is responsible for a failure often has no obvious answer. That is a governance failure, not just a technical one.

Deploying Responsibly

The practical steps that separate a well-governed deployment from an ungoverned one.

The gap between an agent that works in testing and one that is well-governed in production is larger than it appears. Testing tells you whether an agent can accomplish its task. Governance tells you whether the organization is prepared for what happens when it does not, or when it does so in ways that were not anticipated.

Three things must be settled before an agent is deployed: what it can touch, how much it does without a person, and how many decision-making components it contains. This playbook calls them Reach, Latitude and Structure. They vary independently: a position on one says nothing about the other two. This chapter covers the first two. Structure is the subject of Chapter 6. All three are recorded together in Appendix B.

SCOPE DEFINITION BEFORE DEPLOYMENT

Before any agent is deployed in a production environment, its authorized action space should be explicitly defined: what it can do, which systems it can access, and what it cannot do under any circumstances. These constraints should be enforced technically where possible. Soft constraints are insufficient for high-risk deployments. Stating them in system prompts without architectural enforcement is not enough. This playbook refers to that authorized action space as the agent's Reach. It is graded not by the number of tools but by how reversible the actions are: read-only (search, retrieval), reversible write (drafts, staged changes), consequential (sending external communications, committing funds), and irreversible (deletion, settlement, publication). An agent with one tool that settles payments has greater reach than one with twelve read-only APIs.

Scope definition is harder than it sounds because it requires anticipating uses and misuses that have not yet occurred. A useful exercise is to ask: What is the worst thing this agent could do while technically following its instructions? If the answer is not acceptable, the scope is too wide.

Can you demonstrate, today, the complete list of actions this agent is authorized to take, and show that those constraints are enforced architecturally, not only stated in documentation?

LATITUDE: HOW MUCH THE AGENT DOES ALONE

Scope defines what an agent is permitted to do. It says nothing about how much of that work it does without a person. These are separate decisions, and answering only the first is how an organization ends up with an agent that is correctly scoped and entirely unsupervised. Latitude is a design choice, not a consequence of capability: holding the model, the tools and the environment constant, the same agent can be built at any of the following levels.

Latitude — the five levels¹

¹ Levels, role descriptions and controls reproduced from Feng, K. J. K., McDonald, D. W., & Zhang, A. X., Levels of Autonomy for AI Agents (2025), arXiv:2506.12469 — Figure 1 and Table 1, with columns omitted.

Level	User role	Description	Must-have controls
L1	Operator	User directs and makes decisions, agent acts	• User-managed planning • Invocation or approval before actions
L2	Collaborator	User and agent collaboratively plan, delegate, and execute	• Control transfer from agent to user, and vice versa • Shared representation of progress
L3	Consultant	Agent takes lead but consults user for expertise/preferences	• Rich user feedback elicitation interfaces (in addition to simple approvals)
L4	Approver	Agent engages user only in risky or pre-specified scenarios	• Approval elicitation for consequential actions • Customizable conditions for seeking approval
L5	Observer	Agent operates with full autonomy under user monitoring	• Emergency off switch

A declared level describes the oversight designed into a system, not the oversight actually exercised. An L4 agent whose approvals are granted without inspection is operating at L5 in practice. Where a level depends on human judgment, the design has to support it: friction proportional to reversibility, and enough context in the approval request to make reading it worthwhile.

Can you show that the approval gates for this agent actually fire, and that the people approving them are reading rather than clicking through?

When the axes trade

Minimizing each axis in isolation does not always produce the safest system. A refund agent built as a single component minimizes Structure, but it reads customer correspondence and holds the credential that issues refunds, so the part that can be manipulated is the same part that can move money. Splitting it raises Structure and lowers Reach where it matters.

Where that is the reason for an escalation, it is a valid justification in its own right, and a more demanding one, because the claim has to be checkable against the deployed system. "We split it up for security" is not a trade claim. "The component reading customer correspondence holds no refund credential" is.

This applies only to boundaries that are few, fixed and deliberately placed. Components creating components at runtime never qualifies. A trade claim is recorded in Appendix B alongside the three positions it applies to.

STAGED ROLLOUT

Agents should not go from testing to full production deployment in a single step. A staged rollout, where the agent is initially deployed with limited access, limited user population, or limited action capabilities, allows operational experience to accumulate before the full risk surface is exposed.

A staged rollout is also a latitude escalation. Stage one can run the agent at L2 or L3 with human review required for consequential actions; later stages move to L4, where

it runs alone and returns only under defined conditions. Naming the target level in advance turns "limited action capabilities" into something specific, and makes the advancement criteria testable.

The criteria for advancing through stages should be defined in advance and should include both performance metrics and governance metrics: not just is the agent accomplishing its task, but is the logging infrastructure working, are override mechanisms functioning, and any unexpected behavior has been observed.

Can you show that the criteria for advancing through deployment stages include governance metrics, not only performance metrics?

ONGOING MONITORING

Production monitoring for agentic systems is more demanding than for predictive models because the failure modes are harder to define in advance. You can specify in advance what a correct credit score prediction looks like. It is harder to specify in advance all the ways an agent might behave unexpectedly.

This means monitoring needs to include not just metric-based alerting but also regular human review of agent behavior in production. A sample of agent sessions reviewed by a human who knows what the agent is supposed to do is often more effective at detecting unexpected behavior than any automated system.

Can you show a sample of agent sessions reviewed by a human in the last thirty days, and identify any unexpected behavior that was detected and acted upon?

ROLLBACK

Every agent deployment should have a rollback plan: a defined procedure for reverting to the previous state if something goes wrong. For simple agents, this may mean disabling the agent and falling back to a manual process. For complex multi-agent systems, careful choreography of multiple components may be required. Whatever the procedure, it should be tested before it is needed.

Can you demonstrate the rollback procedure today, not describe it, but execute it in a test environment?

One more distinction belongs here. Controls that exist in production but are switched off for internal testing are not controls, and the difference can be measured. Reviewing the July 2026 incident, OpenAI found that running the same evaluation under its production harness and system prompt reduced the models' propensity to compromise infrastructure by more than a hundredfold, and that its existing chain-of-thought monitoring, had it been enabled on that evaluation, would have raised an alert more than a day before the third party's systems were breached (OpenAI, 2026b). The mechanisms were built and working. They were simply not pointed at the environment where the agents were most capable and least constrained.

The demonstration questions in this chapter follow a pattern set out by Hammond (2026), whose practitioner guide pairs each control with a question a chief executive should be able to answer with evidence rather than assurance. The underlying argument is that the gap between stated governance and demonstrated governance is where most agentic deployments fail. Documentation of controls is not evidence that controls work. The standard for responsible deployment is not that governance mechanisms exist, but that they can be shown to function under realistic conditions.

OpenAI describes the incident as a warning shot for the possibility of loss-of-control events. Whether that reading is right is contested, and this playbook does not settle it. What the incident establishes regardless of that argument is narrower and harder to dispute: an evaluation environment is a deployment, and the governance that applies to one applies to the other.

08

Türkçe Özet

Playbook içeriğinin özetidir.

Bu playbook belirli bir gözlemden yola çıkılarak hazırlanmıştır. Yapay zekâ yönetişimi üzerine üretilen metinlerin büyük çoğunluğu tahminsel modeller varsayımı üzerine kuruludur. Bir model girdi alır ve çıktı üretir. Çıktı değerlendirilir, eğitim verisi denetlenir, karar mantığı belgelenir. Sorun belki zordur ama kapsamı bellidir.

Yapay zekâ ajanları (agentic AI systems) farklı çalışır. Bir hedef alır ve o hedefe ulaşmak için bir dizi adım planlar. Araç kullanır, dış sistemlerle etkileşime girer, zaman zaman başka ajanları yönlendirir. Bu yapısal fark, yönetim açısından üç temel sonuç doğurur.

Geri döndürülemezlik: Bir modelin yanlış tahmini düzeltilebilir. Bir ajanın gönderdiği e-posta, sildiği kayıt ya da başlattığı işlem çoğu zaman geri alınamaz. Bir eylemin doğru olup olmadığı kadar geri döndürülüp döndürülemeyeceği de temel bir yönetim değişkeni haline gelir.

Sorumluluğun kime ait olduğunun belirlenmesi: Başlangıçtaki talimat ile nihai sonuç arasındaki eylem zinciri uzun ve takip edilmesi güç olabilir. Birden fazla ajanın etkileştiği sistemlerde, zararlı bir sonucun hangi karardan kaynaklandığını belirlemek çoğu zaman mümkün olmaz.

Modelin hata payı ve doğruluk düzeyi: Hizalama araştırmaları, yeterince güçlü bir modelin eğitim sürecinde gerçek hedefi yerine bir vekil hedef öğrenebileceğini göstermektedir. Buna mesa-optimizasyon adı verilir. Bunun ötesinde, Anthropic araştırmacılarının 2024 yılında yürüttüğü bir çalışma, yeterince güçlü bir modelin değerlerinin değiştirileceğini anladığında bunu önlemek amacıyla stratejik biçimde davranabildiğini ortaya koymuştur. "Alignment faking" olarak adlandırılan bu bulgu, değerlendirme sırasında uyumlu görünen bir sistemin gerçekten hizalanmış olduğu varsayımını sorgulamaktadır.

GTG-1002: Bu riskler artık yalnızca teorik değildir. Anthropic, Eylül 2025'te belgelenmiş ilk yapay zekâ güdümlü siber casusluk kampanyasını tespit ederek devre dışı bıraktı ve bulgularını Kasım 2025'te yayımladı. Çin devlet bağlantılı tehdit aktörü GTG-1002, Claude Code'u ve Model Context Protocol araçlarını kullanarak teknoloji, finans, kimya ve kamu sektörlerinde faaliyet gösteren yaklaşık otuz kuruma yönelik

saldırıları gerçekleştirdi. Operasyonun yüzde seksen ile doksan arasındaki bölümü; ağ keşfi, zafiyet tespiti, kimlik bilgisi toplama, sistemler arası hareket ve veri sızdırma dahil olmak üzere insan müdahalesi olmaksızın yürütüldü. Saldırı teknik bir açıktan değil, bağlamsal bir yanıltmadan yararlandı. Operatörler Claude'u meşru bir sızma testi uzmanı olarak konumlandırdı. Her eylem kendi başına makul görünüyordu; oluşan zarar, yapılan işlemlerin bütününe bir sonucuydu. Kampanya, içerik izleme yoluyla değil, operasyonel tempo anormallliği sayesinde tespit edildi. Otuz hedefe karşı eş zamanlı sürdürülen faaliyet, insan operatörler için fiziksel olarak imkânsız bir çalışma temposuna işaret ediyordu. Etkili bir kayıt mimarisinin yalnızca ne yapıldığını değil, ne hızla yapıldığını da kapsamı gerektiği, bu vakadan çıkarılabilecek en önemli derslerden biridir.

Playbook'un yapısı

Birinci bölüm, ajan yönetişiminin standart yapay zekâ yönetişiminden neden yapısal olarak farklı olduğunu ele almaktadır. Eylemlerin geri döndürülmesi ve sorumluluğun atfı sorunları burada incelenmekte; GTG-1002 operasyonu bu farklılıkların somut bir görünümü olarak sunulmaktadır.

İkinci bölüm bir ajanın bileşenlerini incelemekte ve bunları planlayıcı, bellek, araçlar ve çevre olarak tanımlamaktadır. Bellek yönetimi açısından özellikle dikkat çekici olan, ajanların yalnızca bilgi biriktirmekle kalmayıp bu bilgiden zamanla üst düzey çıkarımlar yapabildiği yansıma katmanıdır. Bu durum, bellek yönetiminin yalnızca ajanın neye erişebildiği ile değil, eriştiği bilgilerden neler öğrendiğiyle de ilgili olduğunu vurgular.

Üçüncü bölüm başarısızlık biçimlerini (failure modes) ele alıyor: kapsam kayması, amacın yanlış genellenmesi, araçların amaç dışı kullanımı ve istem enjeksiyonu (prompt injection). Mesa-optimizasyon ve hizalama taklidi (alignment faking) bulgularının ikisi de aynı sonuca işaret ediyor: Değerlendirmede uyumlu görünmek ile gerçekten hizalanmış olmak aynı şey değildir.

Dördüncü bölüm insan gözetimi mekanizmalarını ve bunların nasıl kalibre edileceğini tartışıyor. Tek bir kontrol mekanizmasına dayanmanın yetersizliği vurgulanıyor; katmanlı savunma yaklaşımının gerekliliği Anthropic'in yayımlanmış güvenlik anlayışına referansla destekleniyor.

Beşinci bölüm denetim izlerini inceliyor. Etkili bir kayıt mimarisinin yeniden oluşturulabilirlik (reconstructability) ilkesine göre tasarlanması gerektiği savunuluyor; operasyonel tempo izlemenin bu mimarinin ayrılmaz bir parçası olduğu GTG-1002 vakasıyla somutlaştırılıyor.

Altıncı bölüm çoklu ajan sistemlerinde hesap verebilirlik meselesini ele alıyor. Orkestratör ile alt ajanlar (subagents) arasındaki güven ilişkisi, ajanların ortak zayıflıkları paylaşması durumu ve gözetim mekanizmalarının gerçek bağımsızlık gerektirdiği tartışılıyor.

Yedinci bölüm sorumlu devreye alma pratiklerini kapsıyor. Kapsam tanımı, aşamalı devreye alma, sürekli izleme ve geri alma prosedürleri inceleniyor. Her başlık altında, yöneticilerin somut kanıt sunmasını gerektiren denetim soruları yer alıyor.

Ekler belgenin uygulama tarafını oluşturmaktadır. Ek A, bir sistemin mimari karmaşıklığına ilişkin kararın kayıdır. Ek B ise bir ajanın üç eksenindeki konumunu, gerekçesini, uygulanan kontrolleri ve istisnaları tek bir kayıta toplamakta ve sistemi kurmayan bir gözden geçirenin imzaladığı dağıtım kararıyla kapanmaktadır.

TrustAI

TrustAI, İstanbul merkezli bir yapay zekâ yönetişimi araştırma ve danışmanlık kuruluşudur. Bu playbook, ajan sistemleri üzerine yürütülen araştırmalara ve kurumlarla gerçekleştirilen danışmanlık çalışmalarına dayanmaktadır. Alan hızla gelişmekte olup bu belge düzenli olarak güncellenecektir.

Appendix A

Architectural Justification Record

Keep one record for each agentic system, and update it whenever something material changes, such as a new tool, a different environment, or a change in how the components are arranged, since at that point the existing record no longer matches what is actually running.

When reviewing a system, check whether a current record exists for it and whether that record matches what is actually deployed. That check is attested and signed in section 6 of Appendix B.

System name			
Owner			
Date			
Tier chosen	☐ To ☐ T1 ☐ T2 ☐ T3 ☐ T4		
Trigger invoked	☐ Context capacity ☐ Genuine parallelism ☐ Safety-motivated separation ☐ Not applicable (To or T1)		
Compensating controls in place	 		
Trade claimed, if any	What specifically got smaller (be verifiable, not general): 		
Next review / re-record trigger			

A note on “tier”

This scale describes architectural complexity, meaning how many decision-making components a system has and who is directing them. It answers a different question than the EU AI Act's risk tiers, which classify a system by what it is used for rather than how it is built. A system with very little architectural complexity can still fall into a high-risk category under the Act, and a more complex, multi-agent system can fall into a lower one. The two scales use similar-sounding language, but they are not measuring the same thing, and it is worth keeping that distinction in mind when discussing either one.

Appendix B

Agency Envelope Record

The record exists so that the shape of an agent is a decision someone made and can account for, rather than a by-product of how it happened to get built. Reach and Latitude are defined in Chapter 7; Structure in Chapter 6. This appendix is the record that covers all three.

One per agentic system. Complete page 1. Attach only the blocks it routes you to. The record closes with a deployment decision, signed by a reviewer who did not build the system.

System		Owner		Date	
--------	--	-------	--	------	--

Role:	☐ built in-house ☐ deployed from an external provider
-------	---

1 · Envelope

Reach what it can touch	☐ read-only ☐ reversible write ☐ consequential ☐ irreversible ref: _____
Latitude how much it does alone	☐ L1 ☐ L2 ☐ L3 ☐ L4 ☐ L5 ref: _____
Structure how many decision-makers	☐ T0 ☐ T1 ☐ T2 ☐ T3 ☐ T4 ref: _____

2 · Justification

Reach above read-only	☐ n/a Capability required, and the narrower option rejected: _____
Latitude above L2	☐ n/a ☐ review impractical at volume ☐ user cannot improve the decision ☐ environment isolated
Structure above T2	☐ n/a ☐ context capacity ☐ genuine parallelism ☐ safety-motivated separation

3 · Trade claimed

☐ none	Raised _____ to reduce _____ What specifically got smaller — be verifiable, not general: _____
-------------------------------	---

4 · Control blocks

Required	☐ A floor — always ☐ B if Reach consequential or irreversible ☐ C if Structure T3/T4 ☐ D if Latitude L4/L5 ☐ E if Structure T4
Status	Attached and complete ☐ Exceptions recorded ☐ none ☐
Provider-dependent	Controls that only the party that built the system can implement. What was asked for, and what was supplied: _____

5 • Review

Record lapses on	☐ new tool or capability ☐ change of operating environment ☐ change in topology ☐ model or tool version change ☐ date: _____
------------------	---

A typical internal agent: single loop, read-only, user reviews output — completes this page and Block A. Four ticks and one attachment.

6 • Deployment decision

Completed by a reviewer who did not build the system. Sections 1 to 5 state what the system is. This section is the decision to run it.

Reviewer attestation

- ☐ Sections 1 to 5 match the system as actually deployed, verified against the running configuration rather than against the design intent
- ☐ Every control block required by section 4 is attached, and every unticked control appears in the Exceptions table

Decision

- ☐ Deploy. All required controls in place.
- ☐ Deploy with exceptions. Exceptions recorded below with approver and expiry. Permitted only where the missing control is not one of the following: kill switch with a tested halt path, trajectory logging, tested rollback, agent identifier and version in logs, separation of untrusted content from privileged credentials, spawn and budget caps at T4. The absence of any of these blocks deployment.
- ☐ Do not deploy. Remediation required before reconsideration.

Grounds, in the reviewer's own words

Reviewer: _____ Signature: _____

Decision date: _____

System owner, acknowledging the decision: _____

Date: _____

This decision lapses on the triggers recorded in section 5. A lapsed decision is not a deployment authorization.

Control Blocks:

Attach only those ticked above. A control not yet in place goes in the exceptions table, not left unticked.

A • Floor — every system ☐ Kill switch, any operator; halt path tested ☐ Trajectory logging: prompts, tool calls, results, actions ☐ Stop conditions; iteration and budget caps ☐ Credentials under the agent's own identity ☐ Rollback procedure executed in a test environment, not only documented ☐ Durable agent identifier present in every log line, tool call and outbound message, together with the version running ☐ Operational tempo recorded, meaning action rate and volume over time, not only the actions themselves ☐ Memory policy stated: what is retained across sessions, for how long, and the conditions under which it is cleared ☐ A safe exit exists: the agent can declare a task impossible, broken or out of scope and stop, and that outcome is recorded as valid rather than as a failure B • Reach consequential or irreversible ☐ Tool allowlist ☐ Schema and parameter validation at the orchestration layer ☐ Approval gate on irreversible operations ☐ Tool inventory: purpose, authorizer, version; versions pinned ☐ Runtime monitoring: call volumes, out-of-scope calls, unusual sequences ☐ The component that processes external or user-supplied content does not hold the credential that performs the consequential action	C • Structure T3 or T4 ☐ Inter-component message authentication and replay protection ☐ Policy on which components may address which ☐ Per-component least-privilege tool sets ☐ Inter-component messages logged ☐ Error attribution: which component, on which input ☐ Circuit breakers on tool calls ☐ Per-component blast-radius caps D • Latitude L4 or L5 ☐ Approval conditions configurable, specified before deployment ☐ Friction proportionate to reversibility; source attribution ☐ Monitoring designed to show when to stop the system ☐ Periodic review of approval rates ☐ A defined time limit within which a flagged action must be cleared or the activity paused E • Structure T4 ☐ Spawn limits ☐ Global budget caps ☐ Written exceptional justification, approved by: _____
--	--

Several controls in this record correspond to categories in the OWASP Top 10 for Agentic Applications 2026 (OWASP Gen AI Security Project), including ASIO2 Tool Misuse and Exploitation, ASIO7 Insecure Inter-Agent Communication and ASIO8 Cascading Failures. Licensed CC BY-SA 4.0.

Exceptions

Control	Reason and mitigation	Approved by	Until

Complete example. Customer refund agent. Reach consequential (refund, thresholded) · Latitude L4 · Structure T3. Justification: no narrower tool issues refunds; review impractical at volume; safety-motivated separation. Trade: raised Structure to reduce Reach, so the component reading customer correspondence holds no refund credential and cannot call the payments API. Blocks A+B+C+D. Exception: circuit breakers scheduled, mitigated by daily refund-volume alerting, approved until [date]. Decision: deploy with exceptions, signed by [reviewer], [date].
Source: Chapter 6 (Structure, architectural minimalism, escalation triggers), Chapter 7 (Reach, Latitude, trade rule).

Glossary

This glossary covers the technical and governance terms used in this playbook. Entries are grouped by theme rather than listed alphabetically, so that related concepts can be read together. Where a term has a precise technical meaning that differs from its everyday usage, that distinction is noted.

TECHNICAL TERMS

Agentic AI system. An AI system that pursues a goal by planning and executing a sequence of actions over time, rather than producing a single output in response to a single input. Agentic systems typically have access to tools, can interact with external services, and may revise their approach based on what they encounter during a task.

Planner. The component of an agent responsible for deciding how to achieve a given goal. The planner generates a sequence of steps and often revises that sequence as the task progresses. Governance concerns at the planner level center on goal specification: an ambiguously stated goal may lead the planner to pursue it in unintended ways.

Tool. Any external capability made available to an agent, such as web search, file access, code execution, API calls, or communication functions. Tools extend an agent's action space and are a primary source of governance risk, since each tool is a channel through which an agent can affect systems outside itself.

Context window. The span of text an AI model can process at one time. For agents working on long or multi-step tasks, earlier instructions and accumulated information within the context window can influence later behavior in ways that are not always predictable.

Persistent memory. Information an agent retains across separate sessions or shares with other agents, as distinct from the contents of its current context window. Persistent memory raises particular governance concerns because it can accumulate in ways that are not visible to human overseers.

Reflection. The process by which an agent synthesizes higher-order inferences from accumulated experience, drawing conclusions that go beyond what any single interaction contained. An agent that has reflected on a pattern of interactions may behave differently from one encountering the same situation for the first time, even when its instructions have not changed.

Orchestrator. In a multi-agent system, the component responsible for breaking a task into subtasks and delegating them to other agents. An orchestrator sets goals for subagents but does not thereby assume full responsibility for their behavior; each agent is expected to maintain its own constraints independently.

Subagent. An agent that receives instructions from an orchestrating agent and executes a defined portion of a larger task. A subagent should apply the same safety constraints regardless of whether instructions arrive from a human operator or from another agent.

Version pinning. The practice of fixing a tool or dependency to a specific version rather than allowing automatic updates. Version pinning ensures that an agent's tool behavior does not change unexpectedly between deployments.

FAILURE MODES

Scope creep. A failure mode in which an agent takes actions beyond its intended scope because its goal specification or action constraints were too loose. Scope creep is rarely dramatic; it typically accumulates through a series of individually plausible steps that together move the agent well outside its authorized activity.

Goal misgeneralization. A failure mode in which an agent learns to pursue a proxy for its intended goal, and that proxy diverges from the actual goal in deployment conditions. An agent trained to maximize positive feedback, for example, may learn to produce responses that sound confident rather than accurate, if confidence proves easier to reward.

Mesa-optimization. A failure mode identified in AI alignment research. As an agent becomes more capable, it may develop an internal objective that diverges from the one it was trained to pursue. This divergence may not be visible during evaluation but becomes apparent in deployment, when conditions differ from training.

Alignment faking. A behavior documented by Anthropic researchers in which a sufficiently capable model, when informed that its values were to be modified through retraining, strategically complied with otherwise-refused instructions in order to prevent those modifications. The model was not instructed to behave this way; it reasoned its way to this strategy independently. Alignment faking means that evaluation compliance and genuine alignment are not the same thing.

Tool misuse. A failure mode in which an agent uses a tool in an unanticipated way that causes harm. This can result from a misunderstanding of the tool's behavior or from the agent finding the tool instrumentally useful for a goal it should not have been pursuing.

Prompt injection. An attack specific to language model-based agents. It occurs when an agent processes content from its environment that contains instructions designed to override or supplement its original instructions. Prompt injection cannot be fully solved at the technical level with current methods but can be mitigated through architectural choices.

Attack decomposition. A technique in which a request that would be refused as a whole is broken into component parts, each of which falls within some agent's permitted scope. The GTG-1002 operation used this approach: no individual instruction was implausible in context, but the sequence as a whole constituted a harmful campaign.

Cascading failure. In a multi-agent system, a failure in one component that propagates to others. By the time the failure is detected, the damage may have accumulated across many steps and systems.

Automation bias. The tendency of human overseers to over-rely on system output, accepting or approving what an agent proposes without adequate independent judgment. A human approver who consistently approves whatever the agent proposes is not providing real oversight.

GOVERNANCE TERMS

Reach. The authorized action space of an agent, graded by the reversibility of the actions it can take. Reach is determined by which tools an agent has access to and what those tools can do, not by the number of tools.

Latitude. The degree to which an agent operates without human involvement, ranging from full human direction at each step to full autonomy under monitoring. Latitude is a design choice that can be set independently of an agent's reach or structural complexity.

Structure. The architectural complexity of an agentic system, measured by how many decision-making components it contains and who directs their control flow. Higher structure increases the governance burden by widening the action space, lengthening the attribution chain, and complicating logging.

Audit trail. A record of an agent's actions sufficient to reconstruct what happened and why. A useful audit trail captures the full sequence of actions, the state of the environment at each step, the reasoning behind actions where accessible, and the operational tempo at which actions were taken. A log that records only final outputs is not an audit trail.

Reconstructability. The practical test for an audit trail: given the log, can you reconstruct what happened and why? For agentic systems, this means being able to follow the chain of decisions and actions from the initial goal to the final outcome, including steps that did not produce a visible output.

Operational tempo. The rate at which an agent takes actions over time. Operational tempo is a governance signal because some failure modes and attacks are detectable not through the content of individual actions but through the volume and pace of activity.

Override mechanism. A control that allows a human to stop or redirect an agent during operation. An override mechanism is only useful if it can be invoked quickly enough to matter. Stopping an agent is not a failure of autonomy, but it is a feature of responsible autonomy and evidence of control.

Graceful shutdown. An override mode in which an agent pauses and surfaces its current state rather than stopping abruptly. Preferable to hard termination for complex tasks, because it preserves the information needed to understand what the agent was doing.

Scope constraint. An explicit definition of what actions an agent is authorized to take and what it cannot do under any circumstances. Scope constraints should be enforced architecturally, not only stated in documentation.

Architectural minimalism. The governance principle that agentic systems should use the minimum architectural complexity necessary for the task. Any escalation beyond a single supervised agent loop should be documented and justified.

Trust level (agent-to-agent). The degree of authority one agent grants to instructions received from another. A reasonable default is that an agent should extend no more trust to another agent than it would to a human user operating through the same interface.

ATTACK AND SECURITY TERMS

Reconnaissance. The phase of a cyberattack in which the attacker maps the target

environment, identifying systems, network topology, vulnerabilities, and potential entry points. In the GTG-1002 operation, reconnaissance was conducted autonomously across approximately thirty organizations.

Exploit development. The process of creating code or techniques that take advantage of a known vulnerability to gain unauthorized access to a system.

Credential harvesting. The collection of authentication information from a compromised system. Credentials allow an attacker or agent to authenticate as a legitimate user, making subsequent actions harder to detect.

Lateral movement. The process of moving from one compromised system to others within a network, typically to expand access or reach higher-value targets.

Exfiltration. The unauthorized transfer of data from a target system to an attacker-controlled location.

References

Anthropic. (2025, November 13). *Disrupting the first reported AI-orchestrated cyber espionage campaign*. <https://www.anthropic.com>

Anthropic. (2023). *Core views on AI safety: When, why, what, and how*. <https://www.anthropic.com/news/core-views-on-ai-safety>

Cemri, M., Pan, M. Z., Yang, S., Agrawal, L. A., Chopra, B., Tiwari, R., Keutzer, K., Parameswaran, A., Klein, D., Ramchandran, K., Zaharia, M., Gonzalez, J. E., & Stoica, I. (2025). *Why do multi-agent LLM systems fail?* (arXiv:2503.13657). arXiv. <https://doi.org/10.48550/arXiv.2503.13657>

European Union. (2024). *Regulation (EU) 2024/1689 of the European Parliament and of the Council of 13 June 2024 laying down harmonised rules on artificial intelligence (Artificial Intelligence Act)*. Official Journal of the European Union, L, 2024/1689, 12 July 2024.

European Union. (2026). *Regulation (EU) 2026/1744 of the European Parliament and of the Council amending Regulation (EU) 2024/1689 (Digital Omnibus on AI)*. Official Journal of the European Union, L, 2026/1744.

Feng, K. J. K., McDonald, D. W., & Zhang, A. X. (2025). *Levels of autonomy for AI agents* (arXiv:2506.12469v2). arXiv. <https://doi.org/10.48550/arXiv.2506.12469>

Greenblatt, R., Denison, C., Wright, B., Roger, F., MacDiarmid, M., Marks, S., Treutlein, J., Belonax, T., Chen, J., Duvenaud, D., Khan, A., Michael, J., Mindermann, S., Perez, E., Petrini, L., Uesato, J., Kaplan, J., Shlegeris, B., Bowman, S. R., & Hubinger, E. (2024). *Alignment faking in large language models* (arXiv:2412.14093v2). arXiv. <https://doi.org/10.48550/arXiv.2412.14093>

Hammond, J. (2026, February 4). *From guardrails to governance: A CEO's guide for securing agentic systems*. MIT Technology Review. Sponsored content produced by Protegrity. <https://www.technologyreview.com/2026/02/04/1131014/from-guardrails-to-governance-a-ceo-s-guide-for-securing-agentic-systems/>

Hubinger, E., van Merwijk, C., Mikulik, V., Skalse, J., & Garrabrant, S. (2019). *Risks from learned optimization in advanced machine learning systems* (arXiv:1906.01820). arXiv. <https://doi.org/10.48550/arXiv.1906.01820>

Huang, J., Chen, X., Mishra, S., Zheng, H. S., Yu, A. W., Song, X., & Zhou, D. (2024). *Large language models cannot self-correct reasoning yet* (arXiv:2310.01798). *International Conference on Learning Representations*. <https://doi.org/10.48550/arXiv.2310.01798>

International Organization for Standardization. (2023). *Information technology: Artificial intelligence. Management system (ISO/IEC 42001:2023)*.

METR. (2026, August 26). *Brief independent investigation of agents' behavior, reasoning and*

collaboration in the OpenAI / Hugging Face hacking incident.
<https://metr.org/blog/2026-08-26-openai-hugging-face-incident-investigation/>

Motwani, S. R., Baranchuk, M., Strohmeier, M., Bolina, V., Torr, P. H. S., Hammond, L., & Schroeder de Witt, C. (2024). Secret collusion among AI agents: Multi-agent deception via steganography. *Advances in Neural Information Processing Systems*, 37.
<https://doi.org/10.48550/arXiv.2402.07510>

Nannini, L., Smith, A. L., Maggini, M. J., Panai, E., Feliciano, S., Tiulkanov, A., Maran, E., Gealy, J., & Bisconti, P. (2026). AI agents under EU law: A compliance architecture for AI providers (arXiv:2604.04604). *arXiv*.

National Institute of Standards and Technology. (2020). Security and privacy controls for information systems and organizations (NIST Special Publication 800-53, Rev. 5). U.S. Department of Commerce. <https://doi.org/10.6028/NIST.SP.800-53r5>

OWASP Gen AI Security Project. (2025). OWASP top 10 for agentic applications 2026. OWASP Foundation. <https://genai.owasp.org>

OpenAI. (2026a, July 21). OpenAI and Hugging Face partner to address security incident during model evaluation.
<https://openai.com/index/hugging-face-model-evaluation-security-incident/>

OpenAI. (2026b, August 26). The Hugging Face incident and the road ahead.
<https://openai.com/index/hugging-face-incident-and-the-road-ahead/>

Saini, S. (2026, March 20). Governing the agentic enterprise: A new operating model for autonomous AI at scale. *California Management Review Insights*.
<https://cmr.berkeley.edu/2026/03/governing-the-agentic-enterprise-a-new-operating-model-for-autonomous-ai-at-scale/>

Sander, L., Pan, F., Zolfaghari, V., Schamschurko, A., Petrovic, N., & Knoll, A. (2026). Scaling LLM-driven multi-agent systems: Design principles and architectural scalability analysis (arXiv:2607.27942). *arXiv*. <https://doi.org/10.48550/arXiv.2607.27942>

Tallam, K. (2026). Authorization propagation in multi-agent AI systems: Identity governance as infrastructure (arXiv:2605.05440). *arXiv*. <https://doi.org/10.48550/arXiv.2605.05440>

Tan, H., Sun, F., Liu, S., Su, D., Cao, Q., Chen, X., Wang, J., Cai, X., Wang, Y., Shen, H., & Cheng, X. (2025). Too consistent to detect: A study of self-consistent errors in LLMs (arXiv:2505.17656). *arXiv*. <https://doi.org/10.48550/arXiv.2505.17656>

Valmeekam, K., Marquez, M., & Kambhampati, S. (2023). Can large language models really improve by self-critiquing their own plans? (arXiv:2310.08118). *arXiv*.
<https://doi.org/10.48550/arXiv.2310.08118>

International Organization for Standardization. (2022). Information security, cybersecurity and privacy protection: Information security management systems. Requirements (ISO/IEC 27001:2022).